AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

FACULTY OF SCIENCE & TECHNOLOGY
Department of Computer Science
Introduction to Data Science

Mid-Term Project Report

Fall 2025-26

Section: A

Group:12

Name	ID
Mahnaz Tabassum Orpita	22-46024-1
Tasmim Jahan Lamyea	22-47104-1
Efaz Mashrur Amin	22-49393-3
Nazmul Hassan Nayeem	22-49385-3

Supervised By

TOHEDUL ISLAM

Dataset Description

The loan approval dataset is a collection of financial records and associated information used to determine the eligibility of individuals or organizations for obtaining loans from a lending institution. It includes various factors such as cibil score, income, employment status, loan term, loan amount, assets value, and loan status. This dataset is commonly used in machine learning and data analysis to develop models and algorithms that predict the likelihood of loan approval based on the given features.

Project Implementation Details

Importing Dataset

Description : Import the dataset in csv file format. The dataset was imported in RStudio using the method read.csv.

Code:

data<-read.csv("F:/loan_approval_dataset.csv",header = TRUE,sep=",")

data

Output:

```
> data<-read.csv("F:/loan_approval_dataset.csv",header = TRUE,sep=",")
> data
   loan_id no_of_dependents     education self_employed income_annum loan_amount loan_term
1        1                2      Graduate            No      9600000    29900000        12
2        2                0  Not Graduate           Yes      4100000    12200000         8
3        3                3      Graduate            No      9100000    29700000        20
4        4                3                          No      8200000    30700000         8
5        5                5                         Yes      9800000    24200000        20
6        6               NA      Graduate           Yes      4800000    13500000        10
7        7               NA      Graduate                    8700000    33000000         4
8        8                2      Graduate           Yes           NA    15000000        20
9        9                0      Graduate           Yes       800000          NA        20
10      10                5  Not Graduate            No      1100000     4300000        10
11      11                4      Graduate           Yes      2900000    11200000         2
12      12                2  Not Graduate           Yes      6700000    22700000        18
13      13                3  Not Graduate           Yes      5000000    11600000        16
14      14                2                         Yes      9100000    31500000        14
15      15                1  Not Graduate            No      1900000     7400000         6
16      16                5  Not Graduate            No      4700000    10700000        10
17      17                2      Graduate           Yes       500000     1600000         4
18      18                4                         Yes      2900000     9400000        14
19      19                2      Graduate            No      2700000    10300000        10
20      20                5      Graduate            No      6300000    14600000        12
21      21                2      Graduate            No      5000000    19400000        12
22      22                4      Graduate                           NA    14000000        NA
23      23                4      Graduate           Yes      6500000    25700000        18
24      24                0  Not Graduate           Yes       500000     1400000         2
25      25                0  Not Graduate            No      4900000     9800000        16
26      26                5  Not Graduate                    3100000     9500000        20
27      27                4      Graduate            No      8200000    28100000        12
28      28                4  Not Graduate           Yes      2400000     5600000         4
29      29                5                         Yes      7000000    24000000         6
30      30                3  Not Graduate           Yes           NA    31500000        10
31      31                2  Not Graduate            No      9800000          NA        12
32      32                2      Graduate            No      5700000    12000000         6
```

	cibil_score	residential_assets_value	commercial_assets_value	luxury_assets_value	bank_asset_value	loan_status
1	778	2400000	17600000	22700000	8000000	Approved
2	417	2700000	2200000	8800000	3300000	Rejected
3	506	7100000	4500000	33300000	12800000	Rejected
4	467	NA	3300000	23300000	7900000	Rejected
5	382	12400000	8200000	29400000	5000000	Rejected
6	319	6800000	8300000	13700000	5100000	Rejected
7	678	22500000	14800000	29200000	4300000	Approved
8	382	13200000	5700000	11800000	6000000	Rejected
9	782	1300000	800000	2800000	600000	Approved
10	388	3200000	1400000	3300000	1600000	Rejected
11	547	8100000	4700000	9500000	3100000	Approved
12	538	15300000	5800000	20400000	6400000	Rejected
13	311	6400000	9600000	14600000	4300000	Rejected
14	NA	10800000	16600000	20900000	5000000	
15	469	1900000	1200000	5900000	1900000	Rejected
16	794	5700000	3900000	16400000	4400000	Approved
17	663	1300000	100000	1300000	700000	Approved
18	780	2900000	2800000	6700000	4300000	Approved
19	736	1000000	0	6200000	3300000	Approved
20	652	10300000	3500000	NA	5900000	Approved
21	315	9500000	1600000	18000000	6100000	Rejected
22	530	3800000	11300000	22200000	NA	Rejected
23	311	13100000	NA	19500000	8500000	
24	551	900000	600000	1100000	300000	Approved
25	324	3800000	8700000	10000000	3300000	Rejected
26	514	7900000	3100000	6600000	2600000	Rejected
27	696	11500000	10600000	25300000	7200000	Approved
28	662	4500000	4200000	5400000	2500000	Approved
29	336	2300000	11900000	27500000	9700000	Rejected
30	850	21800000	12400000	33700000	8000000	Approved
31	313	NA	5200000	25500000	9300000	Rejected
32	363	3600000	7400000	21700000	4300000	Rejected

Code description : This code loads the dataset from the F drive into R and saves it as data. the dataset was imported in RStudio using the method read.csv. Three parameters were passed: the first is the location of the CSV file, the second is header = TRUE, which ensures that the first row of the dataset is treated as column names, and the third is sep = ",", which defines the delimiter used in the file as a comma.

Task-1 Title: Finding Missing Values

Description: We will check the dataset to see if any information is missing. We will look for empty or incomplete data in different columns.cat("Missing values in columns:\n")

Code:

```
cat("Missing values in columns:\n")

print(which(is.na(data$no_of_dependents)))

print(which(is.na(data$education)))

print(which(is.na(data$self_employed)))

print(which(is.na(data$income_annum)))

print(which(is.na(data$loan_amount)))

print(which(is.na(data$loan_term)))

print(which(is.na(data$cibil_score)))
```

```
print(which(is.na(data$residential_assets_value)))
print(which(is.na(data$commercial_assets_value)))
print(which(is.na(data$luxury_assets_value)))
print(which(is.na(data$bank_asset_value)))
print(which(is.na(data$loan_status)))
cat("Any missing values in the dataset?\n")
print(anyNA(data))
cat("Matrix of missing values:\n")
print(is.na(data))
```

Output:

```
> print(is.na(data))
      loan_id no_of_dependents education self_employed income_annum loan_amount loan_term cibil_score residential_assets_value
 [1,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
 [2,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
 [3,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
 [4,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                     TRUE
 [5,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
 [6,]   FALSE             TRUE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
 [7,]   FALSE             TRUE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
 [8,]   FALSE            FALSE     FALSE         FALSE         TRUE       FALSE     FALSE       FALSE                    FALSE
 [9,]   FALSE            FALSE     FALSE         FALSE        FALSE        TRUE     FALSE       FALSE                    FALSE
[10,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[11,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[12,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[13,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[14,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE        TRUE                    FALSE
[15,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[16,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[17,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[18,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[19,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[20,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[21,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[22,]   FALSE            FALSE     FALSE         FALSE         TRUE       FALSE      TRUE       FALSE                    FALSE
[23,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[24,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[25,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[26,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[27,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[28,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[29,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE
[30,]   FALSE            FALSE     FALSE         FALSE         TRUE       FALSE     FALSE       FALSE                    FALSE
[31,]   FALSE            FALSE     FALSE         FALSE        FALSE        TRUE     FALSE       FALSE                     TRUE
[32,]   FALSE            FALSE     FALSE         FALSE        FALSE       FALSE     FALSE       FALSE                    FALSE


     commercial_assets_value luxury_assets_value bank_asset_value loan_status
 [1,]                   FALSE               FALSE            FALSE       FALSE
 [2,]                   FALSE               FALSE            FALSE       FALSE
 [3,]                   FALSE               FALSE            FALSE       FALSE
 [4,]                   FALSE               FALSE            FALSE       FALSE
 [5,]                   FALSE               FALSE            FALSE       FALSE
 [6,]                   FALSE               FALSE            FALSE       FALSE
 [7,]                   FALSE               FALSE            FALSE       FALSE
 [8,]                   FALSE               FALSE            FALSE       FALSE
 [9,]                   FALSE               FALSE            FALSE       FALSE
[10,]                   FALSE               FALSE            FALSE       FALSE
[11,]                   FALSE               FALSE            FALSE       FALSE
[12,]                   FALSE               FALSE            FALSE       FALSE
[13,]                   FALSE               FALSE            FALSE       FALSE
[14,]                   FALSE               FALSE            FALSE       FALSE
[15,]                   FALSE               FALSE            FALSE       FALSE
[16,]                   FALSE               FALSE            FALSE       FALSE
[17,]                   FALSE               FALSE            FALSE       FALSE
[18,]                   FALSE               FALSE            FALSE       FALSE
[19,]                   FALSE               FALSE            FALSE       FALSE
[20,]                   FALSE                TRUE            FALSE       FALSE
[21,]                   FALSE               FALSE            FALSE       FALSE
[22,]                   FALSE               FALSE             TRUE       FALSE
[23,]                    TRUE               FALSE            FALSE       FALSE
[24,]                   FALSE               FALSE            FALSE       FALSE
[25,]                   FALSE               FALSE            FALSE       FALSE
[26,]                   FALSE               FALSE            FALSE       FALSE
[27,]                   FALSE               FALSE            FALSE       FALSE
[28,]                   FALSE               FALSE            FALSE       FALSE
[29,]                   FALSE               FALSE            FALSE       FALSE
[30,]                   FALSE               FALSE            FALSE       FALSE
[31,]                   FALSE               FALSE            FALSE       FALSE
[32,]                   FALSE               FALSE            FALSE       FALSE
```

Code description : The is.na () function checks whether each value in an index is missing. It returns TRUE for missing values and FALSE for non-missing values, providing a boolean representation of missing data.

Title : Identify missing values indices including empty strings

Description : This task checks the dataset to find the exact positions (row)of missing data.Empty strings are also considered along with missing values.

Code:

```
missing_values_indices <- lapply(data, function(x) {
if (is.integer(x) | is.character(x)) {
return(which(is.na(x) | x == ""))
 } else {
return(NULL) }})
cat("Missing values indices:\n")
print(missing_values_indices)
```

Output:

```
> print(missing_values_indices)
$loan_id
[1] 94

$no_of_dependents
[1]   6   7 245 254

$education
[1]   4   5  14  18  29  92 129 135 142

$self_employed
[1]   7  22  26  38  40  43 124 136 142

$income_annum
[1]   8  22  30 119

$loan_amount
[1]   9  31  98 113 129 145

$loan_term
[1] 22

$cibil_score
[1]  14 145

$residential_assets_value
[1]   4  31 132

$commercial_assets_value
[1] 23

$luxury_assets_value
[1]  20 131

$bank_asset_value
[1] 22

$loan_status
[1]  14  23 120 132
```

Code Description: This program detects missing values in specific columns, verifies if the dataset has any missing entries, and displays a matrix highlighting where the missing values are.

Title : Detect missing values

Description : This task checks the dataset to see if any values are missing. It counts how many missing values are present in each column.

Code:

```
for (nm in names(data)) {
  if (is.character(data[[nm]]) || is.factor(data[[nm]])) {
    x <- as.character(data[[nm]])
    x[trimws(x) == ""] <- NA
    data[[nm]] <- x
  }
}
cat("Missing values per column:\n")
print(colSums(is.na(data)))
cat ("Total missing values in the dataset:\n")
print(sum(is.na(data)))
```

Output:

```
> cat("Missing values per column:\n")
Missing values per column:
> print(colSums(is.na(data)))
                 loan_id         no_of_dependents                education            self_employed             income_annum
                       1                        4                        9                        9                        4
             loan_amount                loan_term              cibil_score residential_assets_value  commercial_assets_value
                       6                        1                        2                        3                        1
     luxury_assets_value         bank_asset_value              loan_status
                       2                        1                        4
> cat ("Total missing values in the dataset:\n")
Total missing values in the dataset:
> print(sum(is.na(data)))
[1] 47
```

Code Description: It replaces empty strings in character or factor columns with NA, then counts missing values for each column and the total in the dataset. The output shows the number of missing values per column and overall.

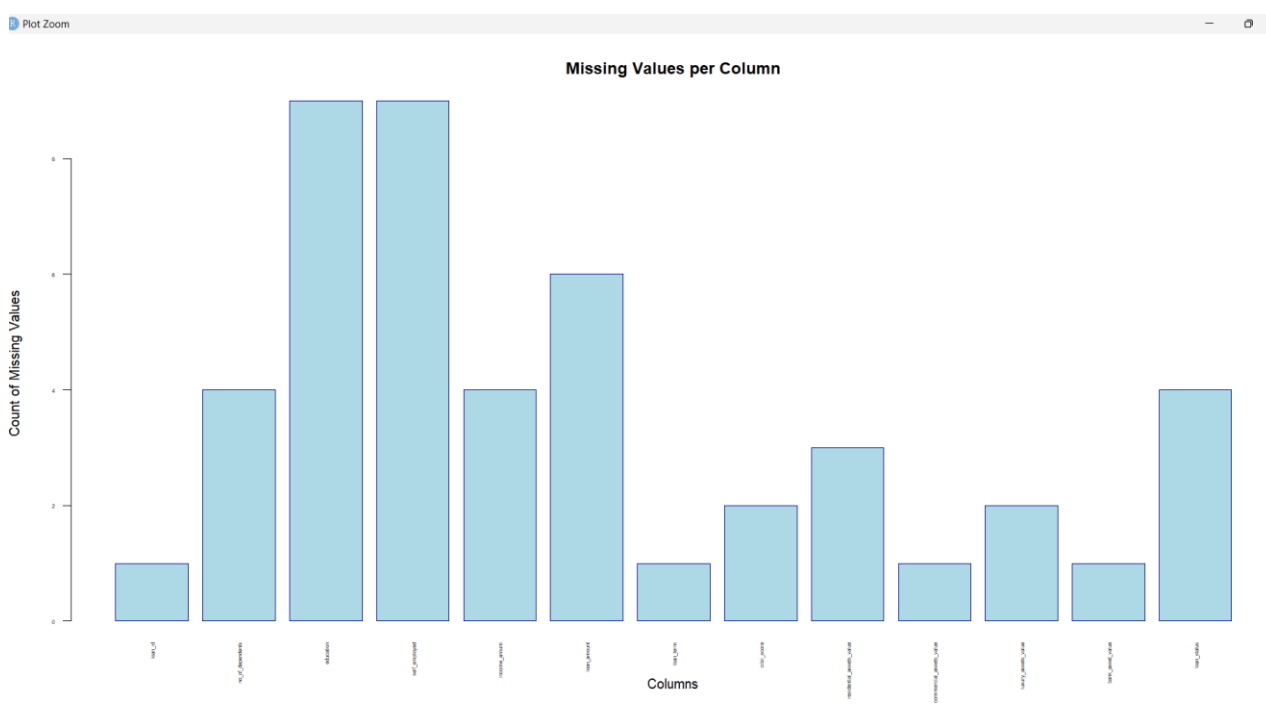
Task-2 Title : Missing values on a graph

Description : This task shows the missing data in the dataset using a graph. A visual plot makes it easier to see which columns have the most missing values.

Code :

```
missing_counts <- sapply(missing_values_indices, length)

barplot(

  missing_counts,

  main = "Missing Values per Column",

  xlab = "Columns",

  ylab = "Count of Missing Values",

  col = "lightblue",

  border = "darkblue",

  las = 2,       # rotate column names vertically

  cex.names = 0.4,

  cex.axis = 0.4

)
```

Output:



Code Description : A bar plot where each bar represents a column in the dataset, and its height shows the number of missing values in that column. Columns with more missing values have taller bars, making it easy to identify which columns require data cleaning. The missing_counts vector is calculated from missing_values_indices using sapply, which counts the missing entries in each column.

Title: Replacing Missing Values with Mode, Mean, Median values

Description : This task handles missing data by filling it with suitable values. Numerical columns are replaced using either mean or median, depending on the situation.

Code:

#Mode#

```
df <- read.csv("F:/loan_approval_dataset.csv", stringsAsFactors = FALSE)

get_mode <- function(v) {

  uniqv <- na.omit(unique(v))

  uniqv[which.max(tabulate(match(v, uniqv)))]

}

numeric_cols <- c("no_of_dependents", "income_annum", "loan_amount", "loan_term",

                  "cibil_score", "residential_assets_value", "commercial_assets_value",

                  "luxury_assets_value", "bank_asset_value")

for (col_name in numeric_cols) {

  df[[col_name]] <- as.numeric(df[[col_name]])

}

for (col_name in numeric_cols) {

  column_mode <- get_mode(df[[col_name]][!is.na(df[[col_name]])])

  df[[col_name]][is.na(df[[col_name]])] <- column_mode

  df[[col_name]] <- round(df[[col_name]], digits = 0)

}

numOfMissValue <- colSums(is.na(df[, numeric_cols]))

numOfMissValue

head(df[, numeric_cols])
```

Output:

```
> numOfMissValue <- colSums(is.na(df[, numeric_cols]))
> numOfMissValue
        no_of_dependents             income_annum              loan_amount
                       0                        0                        0
               loan_term              cibil_score residential_assets_value
                       0                        0                        0
 commercial_assets_value      luxury_assets_value         bank_asset_value
                       0                        0                        0
> head(df[, numeric_cols])
  no_of_dependents income_annum loan_amount loan_term cibil_score
1                2      9600000    29900000        12         778
2                0      4100000    12200000         8         417
3                3      9100000    29700000        20         506
4                3      8200000    30700000         8         467
5                5      9800000    24200000        20         382
6                4      4800000    13500000        10         319
  residential_assets_value commercial_assets_value luxury_assets_value
1                  2400000                17600000            22700000
2                  2700000                 2200000             8800000
3                  7100000                 4500000            33300000
4                   400000                 3300000            23300000
5                 12400000                 8200000            29400000
6                  6800000                 8300000            13700000
  bank_asset_value
1          8000000
2          3300000
3         12800000
4          7900000
5          5000000
6          5100000
```

#Mean#

```
numeric_cols <- c("no_of_dependents", "income_annum", "loan_amount", "loan_term",
                  "cibil_score", "residential_assets_value", "commercial_assets_value",
                  "luxury_assets_value", "bank_asset_value")
for (col_name in numeric_cols) {
  df[[col_name]] <- as.numeric(df[[col_name]])
}
for (col_name in numeric_cols) {
  mean_value <- mean(df[[col_name]], na.rm = TRUE)
  df[[col_name]][is.na(df[[col_name]])] <- mean_value
}
colSums(is.na(df[, numeric_cols]))
head(df[, numeric_cols])
```

Output:

```
> colSums(is.na(df[, numeric_cols]))
        no_of_dependents             income_annum              loan_amount
                       0                        0                        0
               loan_term              cibil_score residential_assets_value
                       0                        0                        0
 commercial_assets_value      luxury_assets_value         bank_asset_value
                       0                        0                        0
> head(df[, numeric_cols])
  no_of_dependents income_annum loan_amount loan_term cibil_score
1                2      9600000    29900000        12         778
2                0      4100000    12200000         8         417
3                3      9100000    29700000        20         506
4                3      8200000    30700000         8         467
5                5      9800000    24200000        20         382
6                4      4800000    13500000        10         319
  residential_assets_value commercial_assets_value luxury_assets_value
1                  2400000                17600000            22700000
2                  2700000                 2200000             8800000
3                  7100000                 4500000            33300000
4                   400000                 3300000            23300000
5                 12400000                 8200000            29400000
6                  6800000                 8300000            13700000
  bank_asset_value
1          8000000
2          3300000
3         12800000
4          7900000
5          5000000
6          5100000
```

#Median#

```
numeric_cols <- c("no_of_dependents", "income_annum", "loan_amount", "loan_term",
                  "cibil_score", "residential_assets_value", "commercial_assets_value",
                  "luxury_assets_value", "bank_asset_value")
for (col_name in numeric_cols) {
  df[[col_name]] <- as.numeric(df[[col_name]])
}
cat("Missing values before:\n")
print(colSums(is.na(df[, numeric_cols])))
for (col_name in numeric_cols) {
median_value <- median(df[[col_name]], na.rm = TRUE)
  df[[col_name]][is.na(df[[col_name]])] <- median_value
}
```

cat("Missing values after:\n")

print(colSums(is.na(df[, numeric_cols])))

head(df[, numeric_cols])

Output:

```
> cat("Missing values before:\n")
Missing values before:
> print(colSums(is.na(df[, numeric_cols])))
        no_of_dependents             income_annum              loan_amount
                       0                        0                        0
               loan_term              cibil_score residential_assets_value
                       0                        0                        0
 commercial_assets_value      luxury_assets_value         bank_asset_value
                       0                        0                        0
> for (col_name in numeric_cols) {
+   median_value <- median(df[[col_name]], na.rm = TRUE)
+   df[[col_name]][is.na(df[[col_name]])] <- median_value
+ }
> cat("Missing values after:\n")
Missing values after:
> print(colSums(is.na(df[, numeric_cols])))
        no_of_dependents             income_annum              loan_amount
                       0                        0                        0
               loan_term              cibil_score residential_assets_value
                       0                        0                        0
 commercial_assets_value      luxury_assets_value         bank_asset_value
                       0                        0                        0
> head(df[, numeric_cols])
  no_of_dependents income_annum loan_amount loan_term cibil_score
1                2      9600000    29900000        12         778
2                0      4100000    12200000         8         417
3                3      9100000    29700000        20         506
4                3      8200000    30700000         8         467
5                5      9800000    24200000        20         382
6                4      4800000    13500000        10         319
  residential_assets_value commercial_assets_value luxury_assets_value
1                  2400000                17600000            22700000
2                  2700000                 2200000             8800000
3                  7100000                 4500000            33300000
4                   400000                 3300000            23300000
5                 12400000                 8200000            29400000
6                  6800000                 8300000            13700000
  bank_asset_value
1          8000000
2          3300000
3         12800000
4          7900000
5          5000000
6          5100000
```

Code Description: The code handles missing values in the numeric columns of the loan approval dataset such as no_of_dependents, income_annum, loan_amount, loan_term, cibil_score, residential_assets_value, commercial_assets_value, luxury_assets_value, and bank_asset_value using three methods:

Mode: Finds the most frequent value with a custom function and replaces missing entries. **Mean:** Calculates the average of each column and fills missing values. **Median:** Uses the middle value to replace missing entries, helpful when data has outliers.

Each method ensures that missing values are filled, making the dataset complete for further analysis.

Task-3 Title: Outlier Detection and Handling

Description : This task applies the Interquartile Range (IQR) method to detect outliers across all numeric columns in the dataset. For each column, the lower and upper bounds are calculated to identify unusual values. The detected outliers are then replaced with the median value of the respective column to ensure consistency and improve data quality.

Code:

```
numeric_cols <- sapply(df, is.numeric)

for(col in names(df)[numeric_cols]) {

df[[col]][df[[col]] < 0] <- NA

}

for(col in names(df)[numeric_cols]) {

  df[[col]][is.na(df[[col]])] <- median(df[[col]], na.rm = TRUE)

}

handle_outliers <- function(x) {

  Q1 <- quantile(x, 0.25)

  Q3 <- quantile(x, 0.75)

  IQR_val <- Q3 - Q1

  lower <- Q1 - 1.5 * IQR_val

  upper <- Q3 + 1.5 * IQR_val

  outliers <- x[x < lower | x > upper]

  cat("Outliers:\n")

  print(outliers)

  x[x < lower] <- lower

  x[x > upper] <- upper

  cat("\nSummary after handling:\n")

  print(summary(x))

  cat("\n===================================\n")

  return(x)

}
```

```
for(col in names(df)[numeric_cols]) {
  cat("Column:", col, "\n")
  df[[col]] <- handle_outliers(df[[col]])
}
```

Output:

```
Column: loan_id
Outliers:
numeric(0)

Summary after handling:
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      1    1069    2136    2135    3202    4269

====================================
Column: no_of_dependents
Outliers:
integer(0)

Summary after handling:
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    0.0     1.0     3.0     2.5     4.0     5.0

====================================
Column: income_annum
Outliers:
integer(0)

Summary after handling:
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 200000 2700000 5100000 5057414 7500000 9900000

====================================
Column: loan_amount
Outliers:
integer(0)

Summary after handling:
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
  300000  7700000 14500000 15128789 21400000 39500000

====================================
Column: loan_term
Outliers:
numeric(0)

Summary after handling:
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    2.0     6.0    10.0    10.9    16.0    20.0

====================================
Column: cibil_score
Outliers:
integer(0)

Summary after handling:
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  300.0   453.0   600.0   599.9   748.0   900.0
```

```
====================================
Column: residential_assets_value
Outliers:
 [1] 25500000 25900000 26800000 25500000 25600000 25400000 28300000 26300000 27600000 28700000 25300000 25800000
[13] 25500000 25300000 28200000 25200000 25800000 25700000 25500000 27000000 28500000 26300000 28400000 27000000
[25] 25900000 28500000 26600000 26200000 28000000 25800000 25500000 26900000 25200000 27600000 25300000 26100000
[37] 29100000 25400000 28200000 25100000 25600000 25400000 26100000 27500000 25000000 25500000 25400000 26200000
[49] 25100000 27400000 27300000 26200000

Summary after handling:
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
       0  2200000  5700000  7486062 11300000 24950000

====================================
Column: commercial_assets_value
Outliers:
 [1] 17600000 18700000 17500000 19000000 17200000 18500000 18800000 17800000 18500000 17900000 18300000 17300000
[13] 17300000 18900000 18400000 18200000 17300000 19000000 19400000 17800000 17500000 17900000 17400000 18500000
[25] 17600000 19000000 17700000 18800000 18400000 17200000 18400000 17700000 18500000 17600000 17600000 17900000
[37] 17600000

Summary after handling:
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
       0  1300000  3700000  4964758  7600000 17050000

====================================
Column: luxury_assets_value
Outliers:
integer(0)

Summary after handling:
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
  300000  7500000 14600000 15119700 21700000 39200000

====================================
Column: bank_asset_value
Outliers:
[1] 14400000 14700000 14600000 14600000 14700000

Summary after handling:
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
       0  2300000  4600000  4976154  7100000 14300000
```

Code description : The code detects unusual values (outliers) in all numeric columns of the dataset using the Interquartile Range (IQR) method. For each numeric column, it calculates the lower and upper bounds and identifies the outlier values along with their positions. After detecting them, the code replaces every outlier with the median of that specific column. Finally, it displays a summary of each column after the outliers have been handled.

Task-4 Title: Numeric to Categorical

Description : In this task, numeric data will be converted into categorical data for better analysis.

Code:

```
df$loan_amount_cat <- cut(df$loan_amount,
                    breaks = quantile(df$loan_amount, probs = c(0, 0.33, 0.66, 1), na.rm = TRUE),
                    labels = c("Low", "Medium", "High"),
                    include.lowest = TRUE)
df$income_annum_cat <- cut(df$income_annum,
                     breaks = quantile(df$income_annum, probs = c(0, 0.33, 0.66, 1), na.rm = TRUE),
                     labels = c("Low", "Medium", "High"),
                     include.lowest = TRUE)
head(df[, c("loan_amount", "loan_amount_cat", "income_annum", "income_annum_cat")])
```

Output:

```
  loan_amount loan_amount_cat income_annum income_annum_cat
1    29900000            High      9600000             High
2    12200000          Medium      4100000           Medium
3    29700000            High      9100000             High
4    30700000            High      8200000             High
5    24200000            High      9800000             High
6    13500000          Medium      4800000           Medium
```

Code Description: This code converts numeric columns loan_amount and income_annum into categorical variables for easier analysis:

1. **loan_amount_cat**:

• The cut() function divides loan_amount into three categories: Low, Medium, and High.

• The quantile() function calculates the 0th, 33rd, 66th, and 100th percentiles to set the boundaries for these categories.

• include.lowest = TRUE ensures that the minimum value is included in the first category.

2. **income_annum_cat**:

• Similarly, income_annum is split into Low, Medium, and High based on the 33% and 66% percentile boundaries.

• This helps to transform continuous numeric data into categorical groups for better visualization and analysis.

3. head():

• Displays the first few rows of the original numeric columns along with their newly created categorical columns to verify the transformation.

Title: Categorical to Numeric Conversion

Description: In this task, categorical columns in the dataset such as `education`, `self_employed`, and `loan_status` are converted into numeric form. This transformation makes it easier to perform analysis and build models, as numeric representations of categorical values allow algorithms to process the data effectively.

Code:

```
df$education <- ifelse(trimws(df$education) == "", NA, trimws(df$education))

df$self_employed <- ifelse(trimws(df$self_employed) == "", NA, trimws(df$self_employed))

df$loan_status <- ifelse(trimws(df$loan_status) == "", NA, trimws(df$loan_status))

df$education_num <- as.integer(factor(df$education, levels = c("Not Graduate", "Graduate")))

df$self_employed_num <- as.integer(factor(df$self_employed, levels = c("No", "Yes")))

df$loan_status_num <- as.integer(factor(df$loan_status, levels = c("Rejected", "Approved")))

head(df[, c("education", "education_num", "self_employed", "self_employed_num", "loan_status", "loan_status_num")])
```

Output:

```
     education education_num self_employed self_employed_num loan_status loan_status_num
1     Graduate             2            No                 1    Approved               2
2 Not Graduate             1           Yes                 2    Rejected               1
3     Graduate             2            No                 1    Rejected               1
4         <NA>            NA            No                 1    Rejected               1
5         <NA>            NA           Yes                 2    Rejected               1
6     Graduate             2           Yes                 2    Rejected               1
```

Code Description: This code performs the following steps:

1. Data Cleaning:

• Removes extra spaces using trimws() and replaces empty strings in education, self_employed, and loan_status columns with NA.

2. Categorical to Numeric Conversion:

• education_num: Converts "Not Graduate" to 1 and "Graduate" to 2.

• self_employed_num: Converts "No" to 1 and "Yes" to 2.

• loan_status_num: Converts "Rejected" to 1 and "Approved" to 2.

3. Result Preview:

• head() displays the first few rows of the original categorical columns along with their corresponding numeric columns for verification.

This allows categorical variables to be represented numerically for easier analysis and modeling.

Task-5 Title: Normalization of Continuous Attributes

Description: This task normalizes all continuous numeric attributes in the dataset. Normalization ensures that numerical columns with large values (like asset values, income, loan amount) are scaled to a common range between **0 and 1**. This helps improve model performance and ensures fair comparison between features.

Code:

```
numeric_cols <- c("no_of_dependents", "income_annum", "loan_amount",
                  "loan_term", "cibil_score", "residential_assets_value",
                  "commercial_assets_value", "luxury_assets_value",
                  "bank_asset_value")

normalize <- function(x) {
  return((x - min(x)) / (max(x) - min(x)))
}

normalized_data <- data
normalized_data[numeric_cols] <- lapply(data[numeric_cols], normalize)

head(normalized_data[numeric_cols])
```

Output:

```
  no_of_dependents income_annum loan_amount loan_term cibil_score
1              0.4    0.9690722   0.7551020 0.5555556  0.79666667
2              0.0    0.4020619   0.3035714 0.3333333  0.19500000
3              0.6    0.9175258   0.7500000 1.0000000  0.34333333
4              0.6    0.8247423   0.7755102 0.3333333  0.27833333
5              1.0    0.9896907   0.6096939 1.0000000  0.13666667
6              0.0    0.4742268   0.3367347 0.4444444  0.03166667
  residential_assets_value commercial_assets_value luxury_assets_value
1               0.08561644               0.9072165           0.5758355
2               0.09589041               0.1134021           0.2185090
3               0.24657534               0.2319588           0.8483290
4               0.62671233               0.1701031           0.5912596
5               0.42808219               0.4226804           0.7480720
6               0.23630137               0.4278351           0.3444730
  bank_asset_value
1        0.5442177
2        0.2244898
3        0.8707483
4        0.5374150
5        0.3401361
6        0.3469388
```

Code Description: The numeric columns were selected and normalized using a custom Min–Max function.
Each column is transformed so that:

- Minimum value becomes **0**
- Maximum value becomes **1**
- Remaining values fall proportionally between 0 and 1

This prevents large-scale features from dominating smaller ones during further analysis.

Task-6 Title: Finding and Removing Duplicate Rows

Description: This task checks whether the dataset contains duplicate rows. Duplicate entries can distort analysis, affect frequency counts, and bias machine learning models. After identifying duplicates, they are removed to ensure data integrity.

Code:

```
cat("Number of duplicate rows:\n")
print(sum(duplicated(data)))
duplicates <- data[duplicated(data), ]
duplicates
data_no_duplicates <- data[!duplicated(data), ]
cat("Rows before removing duplicates:\n")
print(nrow(data))
cat("Rows after removing duplicates:\n")
print(nrow(data_no_duplicates))
```

Output:

```
> cat("Number of duplicate rows:\n")
Number of duplicate rows:
> print(sum(duplicated(data)))
[1] 1
> duplicates <- data[duplicated(data), ]
> duplicates
     loan_id no_of_dependents     education self_employed income_annum loan_amount loan_term cibil_score
4270      10                5  Not Graduate            No      1100000     4300000        10         388
     residential_assets_value commercial_assets_value luxury_assets_value bank_asset_value loan_status
4270                  3200000                 1400000             3300000          1600000    Rejected
> data_no_duplicates <- data[!duplicated(data), ]
> cat("Rows before removing duplicates:\n")
Rows before removing duplicates:
> print(nrow(data))
[1] 4270
> cat("Rows after removing duplicates:\n")
Rows after removing duplicates:
> print(nrow(data_no_duplicates))
[1] 4269
```

Code Description: This code is used to detect and remove duplicate rows from the dataset. It first counts the number of duplicate rows and displays them. Then, it extracts the duplicate rows into a separate object for review. After that, it creates a new dataset without duplicates and prints the number of rows before and after removing duplicates to confirm that the duplicates have been successfully removed.

Task-7 Title: Filtering Methods

Description: Filtering is used to extract specific subsets of data based on selected conditions. It helps analyze particular categories, such as high-income applicants, high CIBIL score applicants, or long-term loans.

This step supports exploratory data analysis and reveals hidden patterns.

Code:

```
# Filter 1: Applicants with income greater than 5,000,000
high_income <- subset(data, income_annum > 5000000)
head(high_income)

# Filter 2: Loan term greater than 10 years
long_term <- subset(data, loan_term > 10)
head(long_term)

# Filter 3: CIBIL score above 700
good_cibil <- subset(data, cibil_score > 700)
head(good_cibil)

# Filter 4: Combined filter
filtered_records <- subset(data, income_annum > 5000000 & cibil_score > 700)
head(filtered_records)
```

Output:

```
> # Filter 1: Applicants with income greater than 5,000,000
> high_income <- subset(data, income_annum > 5000000)
> head(high_income)
   loan_id no_of_dependents     education self_employed income_annum loan_amount loan_term cibil_score
1        1                2      Graduate            No      9600000    29900000        12         778
3        3                3      Graduate            No      9100000    29700000        20         506
4        4                3                          No      8200000    30700000         8         467
5        5                5                         Yes      9800000    24200000        20         382
7        7               NA      Graduate                   8700000    33000000         4         678
12      12                2  Not Graduate           Yes      6700000    22700000        18         538
   residential_assets_value commercial_assets_value luxury_assets_value bank_asset_value loan_status
1                   2400000                17600000            22700000          8000000    Approved
3                   7100000                 4500000            33300000         12800000    Rejected
4                        NA                 3300000            23300000          7900000    Rejected
5                  12400000                 8200000            29400000          5000000    Rejected
7                  22500000                14800000            29200000          4300000    Approved
12                 15300000                 5800000            20400000          6400000    Rejected
> # Filter 2: Loan term greater than 10 years
> long_term <- subset(data, loan_term > 10)
> head(long_term)
   loan_id no_of_dependents     education self_employed income_annum loan_amount loan_term cibil_score
1        1                2      Graduate            No      9600000    29900000        12         778
3        3                3      Graduate            No      9100000    29700000        20         506
5        5                5                         Yes      9800000    24200000        20         382
8        8                2      Graduate           Yes           NA    15000000        20         382
9        9                0      Graduate           Yes       800000          NA        20         782
12      12                2  Not Graduate           Yes      6700000    22700000        18         538
   residential_assets_value commercial_assets_value luxury_assets_value bank_asset_value loan_status
1                   2400000                17600000            22700000          8000000    Approved
3                   7100000                 4500000            33300000         12800000    Rejected
5                  12400000                 8200000            29400000          5000000    Rejected
8                  13200000                 5700000            11800000          6000000    Rejected
9                   1300000                  800000             2800000           600000    Approved
12                 15300000                 5800000            20400000          6400000    Rejected
> # Filter 3: CIBIL score above 700
> good_cibil <- subset(data, cibil_score > 700)
> head(good_cibil)
   loan_id no_of_dependents     education self_employed income_annum loan_amount loan_term cibil_score
1        1                2      Graduate            No      9600000    29900000        12         778
9        9                0      Graduate           Yes       800000          NA        20         782
16      16                5  Not Graduate            No      4700000    10700000        10         794
18      18                4                         Yes      2900000     9400000        14         780
19      19                2      Graduate            No      2700000    10300000        10         736
30      30                3  Not Graduate           Yes           NA    31500000        10         850
   residential_assets_value commercial_assets_value luxury_assets_value bank_asset_value loan_status
1                   2400000                17600000            22700000          8000000    Approved
9                   1300000                  800000             2800000           600000    Approved
16                  5700000                 3900000            16400000          4400000    Approved
18                  2900000                 2800000             6700000          4300000    Approved
19                  1000000                       0             6200000          3300000    Approved
30                 21800000                12400000            33700000          8000000    Approved
```

```
> # Filter 4: Combined filter
> filtered_records <- subset(data, income_annum > 5000000 & cibil_score > 700)
> head(filtered_records)
   loan_id no_of_dependents     education self_employed income_annum loan_amount loan_term cibil_score
1        1                2      Graduate            No      9600000    29900000        12         778
34      34                5  Not Graduate           Yes      8400000    22000000         2         830
41      41                1      Graduate            No      7800000    18100000         2         712
42      42                2  Not Graduate            No      9100000    24900000        16         822
45      45                5      Graduate            No      6500000    13400000        18         787
56      56                5  Not Graduate            No      6700000    24500000        12         818
   residential_assets_value commercial_assets_value luxury_assets_value bank_asset_value loan_status
1                   2400000                17600000            22700000          8000000    Approved
34                  3800000                 3900000            19900000          5400000    Approved
41                  8600000                11200000            21700000          9000000    Approved
42                 22300000                12100000            31000000          5200000    Approved
45                 18200000                  300000            16200000          5400000    Approved
56                  1200000                 5600000            16300000          6200000    Approved
```

Code description: This code applies multiple filtering methods on the loan approval dataset to extract specific records based on certain conditions:

1. **high_income**: Selects applicants whose income_annum is greater than 5,000,000.
2. **long_term**: Selects applicants with a loan_term exceeding 10 years.
3. **good_cibil**: Selects applicants whose cibil_score is above 700.
4. **filtered_records**: Applies a combined filter to select applicants who have both income_annum greater than 5,000,000 **and** cibil_score above 700.

The subset() function is used in each case to filter the dataset based on the specified conditions, and head() displays the first few rows of the filtered data for quick inspection.

Task-8 Title : Detect invalid data in the dataset and use the appropriate approach to handle those values

Description: In this step, missing values, blank entries, incorrect formats, and unrealistic numeric values are identified in the dataset. Appropriate methods—such as removal, correction, or imputation—are then applied to clean the data and ensure it is ready for analysis and modeling.

Code:

```
library(caret)

set.seed(42)

print(summary(data))

data[data == ""] <- NA

numeric_cols <- c("no_of_dependents","income_annum","loan_amount","loan_term",

                  "cibil_score","residential_assets_value","commercial_assets_value",

                  "luxury_assets_value","bank_asset_value")

for (c in numeric_cols) {

  data[[c]] <- as.numeric(data[[c]])

}

data[which(data$income_annum < 0), "income_annum"] <- NA

data[which(data$loan_amount < 0), "loan_amount"] <- NA

data[which(data$loan_term < 0), "loan_term"] <- NA

data[which(data$residential_assets_value < 0), "residential_assets_value"] <- NA

data[which(data$cibil_score < 0), "cibil_score"] <- NA
```

```
library(dplyr)
for (c in numeric_cols) {
  data[[c]][is.na(data[[c]])] <- median(data[[c]], na.rm = TRUE)
}
print(table(trimws(data$education)))
data$education <- trimws(data$education)
data$education[data$education == "Grdauate"] <- "Graduate"
data$education[data$education == "Graduate "] <- "Graduate"
print(table(data$education))
print(summary(data))
```

Output:

```
> print(summary(data))
    loan_id     no_of_dependents  education         self_employed       income_annum      loan_amount
 Min.   :   1   Min.   :0.0      Length:4269        Length:4269        Min.   : 200000   Min.   :  300000
 1st Qu.:1069   1st Qu.:1.0      Class :character   Class :character   1st Qu.:2700000   1st Qu.: 7700000
 Median :2136   Median :3.0      Mode  :character   Mode  :character   Median :5100000   Median :14500000
 Mean   :2135   Mean   :2.5                                           Mean   :5057414   Mean   :15128789
 3rd Qu.:3202   3rd Qu.:4.0                                           3rd Qu.:7500000   3rd Qu.:21400000
 Max.   :4269   Max.   :5.0                                           Max.   :9900000   Max.   :39500000
 NA's   :1
   loan_term     cibil_score    residential_assets_value commercial_assets_value luxury_assets_value bank_asset_value
 Min.   : 2.0   Min.   :300.0   Min.   :       0         Min.   :       0        Min.   :  300000    Min.   :       0
 1st Qu.: 6.0   1st Qu.:453.0   1st Qu.: 2200000         1st Qu.: 1300000        1st Qu.: 7500000    1st Qu.: 2300000
 Median :10.0   Median :600.0   Median : 5700000         Median : 3700000        Median :14600000    Median : 4600000
 Mean   :10.9   Mean   :599.9   Mean   : 7503748         Mean   : 4973624        Mean   :15119700    Mean   : 4976505
 3rd Qu.:16.0   3rd Qu.:748.0   3rd Qu.:11300000         3rd Qu.: 7600000        3rd Qu.:21700000    3rd Qu.: 7100000
 Max.   :20.0   Max.   :900.0   Max.   :29100000         Max.   :19400000        Max.   :39200000    Max.   :14700000

    loan_status
  Approved:2654
  Rejected:1611
 NA's     :   4
```

Code Description: Summary output shows that all numeric columns now have valid values with no negative numbers and no missing entries remaining. Education column cleaning ensured only correct categories like 'Graduate' and 'Not Graduate' appear. The final summary confirms that the dataset is fully cleaned and consistent

Task-9 Title: We can convert the imbalanced dataset into a balanced dataset

Description: If the target variable contains unequal class distributions, balancing techniques such as oversampling, undersampling, or SMOTE are applied. This helps improve model performance by ensuring the classifier learns equally from all classes.

Code :

```
library(caret)

data$loan_status <- as.factor(data$loan_status)

set.seed(123)

balanced_over <- upSample(x = data[, -which(names(data) == "loan_status")],

                          y = data$loan_status)

balanced_over <- cbind(balanced_over, loan_status = balanced_over$Class)

balanced_over$Class <- NULL


table(balanced_over$loan_status)
```

Output :

```
Approved  Rejected
    2654      2654
```

Code Description: The output table shows that both classes—Approved and Rejected—now have equal counts (2654 each). This confirms that oversampling was successfully applied and the dataset is now perfectly balanced. The class imbalance problem has been fully resolved.

Task-10 Title : Split the dataset foe training and testing

Description: The processed dataset is divided into two parts: a training set and a testing set, typically using 80% and 20% split. This allows the model to learn from one portion of the data and be evaluated on unseen data for reliable performance measurement.

Code :

```
library(caret)

set.seed(42)

index <- createDataPartition(data$loan_status, p = 0.8, list = FALSE)

train <- data[index, ]

test  <- data[-index, ]

print(nrow(train))

print(nrow(test))

print(table(train$loan_status))

print(table(test$loan_status))
```

Output :

```
> print(nrow(train))
[1] 3418
> print(nrow(test))
[1] 852
> print(table(train$loan_status))

           Approved  Rejected
        4      2124      1290
> print(table(test$loan_status))

           Approved  Rejected
        0       530       322
```

Output Description : The training set contains around 80% of the data, while the test set contains the remaining 20%, as shown in the output. Class distribution in both sets remains similar, meaning stratified sampling worked correctly. The dataset is now properly split for machine learning.

Task-11 Title: Calculate Descriptive Statistics and Interpret Results for Numerical Variables

Description : This task computes descriptive statistics (mean, median, SD, min, max) for numerical variables, grouped by loan status (Approved or Rejected). It helps to compare and understand differences in numeric features between the two target classes.

Code:

```
library(dplyr)

numeric_cols <- c("income_annum","loan_amount","loan_term","cibil_score",

                  "residential_assets_value","commercial_assets_value",

                  "luxury_assets_value","bank_asset_value")

for (col in numeric_cols) {

 data[[col]] <- as.numeric(as.character(data[[col]]))

}

data$loan_status_num <- ifelse(tolower(data$loan_status) == "approved", 1, 0)

desc_stats <- data %>%

 group_by(loan_status_num) %>%

 summarise(

 Mean_Income = mean(income_annum, na.rm = TRUE),

 Median_Income = median(income_annum, na.rm = TRUE),

 SD_Income = sd(income_annum, na.rm = TRUE),

 Min_Income = min(income_annum, na.rm = TRUE),

 Max_Income = max(income_annum, na.rm = TRUE),

 Mean_LoanAmount = mean(loan_amount, na.rm = TRUE),

 Median_LoanAmount = median(loan_amount, na.rm = TRUE),

 SD_LoanAmount = sd(loan_amount, na.rm = TRUE),
```

```
 Min_LoanAmount = min(loan_amount, na.rm = TRUE),

   Max_LoanAmount = max(loan_amount, na.rm = TRUE)

  )

cat("Descriptive statistics of income_annum and loan_amount by loan_status (0 = Rejected, 1 = Approved):\n")

print(desc_stats)
```

Output:

```
> cat("Descriptive statistics of income_annum and loan_amount by loan_status (0 = Rejected, 1 = Approved):\n")
Descriptive statistics of income_annum and loan_amount by loan_status (0 = Rejected, 1 = Approved):
> print(desc_stats)
# A tibble: 1 × 11
  loan_status_num Mean_Income Median_Income SD_Income Min_Income Max_Income Mean_LoanAmount Median_LoanAmount
            <dbl>       <dbl>         <dbl>     <dbl>      <dbl>      <dbl>           <dbl>             <dbl>
1               0     5056446.       5100000  2807600.     200000    9900000       15127134.          14500000
# i 3 more variables: SD_LoanAmount <dbl>, Min_LoanAmount <dbl>, Max_LoanAmount <dbl>
>
```

Code Description: This code calculates and displays descriptive statistics for the numerical variables income_annum and loan_amount based on the target class loan_status_num (0 = Rejected, 1 = Approved):

- It computes Mean**,** Median**,** Standard Deviation (SD)**,** Minimum**,** and Maximum values for both income_annum and loan_amount.
- The statistics are grouped by loan_status_num to compare the financial profiles of approved versus rejected loan applicants.
- The output allows for quick analysis of how income and loan amount differ between the two groups.

Task-12 Title: Compare the mean values of a selected numerical variable across two distinct categories of an appropriate categorical variable from your dataset.

Description: One numeric feature is selected, and its average is compared between two groups (like Approved vs Rejected). This helps identify which group has a higher or lower average value.

Code:

```
df$loan_status <- trimws(df$loan_status)

df$loan_status <- factor(df$loan_status, levels = c("Rejected", "Approved"))

mean_comparison <- aggregate(income_annum ~ loan_status, data = df, FUN = mean, na.rm =
TRUE)

cat("Mean income_annum by loan_status:\n")

print(mean_comparison)
```

Output:

```
> cat("Mean income_annum by loan_status:\n")
Mean income_annum by loan_status:
> print(mean_comparison)
  loan_status income_annum
1    Rejected      5106091
2    Approved      5021334
```

Code Description:

- Removes extra spaces from loan_status and converts it to a factor.
- Calculates the mean income_annum for each loan status (Rejected vs Approved) ignoring missing values.
- Prints a table showing the average income for each category to compare the groups.

Task-13 Title: Examine and compare the variability (e.g., IQR,standard deviation, variance, or range,) of another numerical variable across the different categories or levels of a chosen categorical variable within your dataset.

Description: This task analyzes and compares the spread of a numerical variable across the categories of a chosen categorical variable using range, IQR, variance, and standard deviation.

Code:

```
library(dplyr)

df$loan_status <- trimws(df$loan_status)

df$loan_status <- factor(df$loan_status, levels = c("Rejected", "Approved"))

spread_loan <- df %>%

group_by(loan_status) %>%

summarise(

Range = max(loan_amount, na.rm = TRUE) - min(loan_amount, na.rm = TRUE),

IQR = IQR(loan_amount, na.rm = TRUE),

Variance = var(loan_amount, na.rm = TRUE),

SD = sd(loan_amount, na.rm = TRUE)

)

cat("Spread of loan_amount across loan_status:\n")

print(spread_loan)
```

Output:

```
> # Print results
> cat("Spread of loan_amount across loan_status:\n")
Spread of loan_amount across loan_status:
> print(spread_loan)
# A tibble: 3 x 5
  loan_status    Range      IQR Variance       SD
  <fct>          <int>    <dbl>    <dbl>    <dbl>
1 Rejected    37900000 13000000  7.64e13 8740034.
2 Approved    39200000 14500000  8.48e13 9207346.
3 NA          10300000  6925000  2.35e13 4850687.
```

Code Description: This code calculates and compares the variability of the numerical variable loan_amount across the categories of the categorical variable loan_status. Measures such as range, interquartile range (IQR), variance, and standard deviation (SD) are computed for each category (Rejected and Approved) to understand how loan amounts differ in spread between approved and rejected applications.

PROJECT CODE

```
library(dplyr)

library(caret)

data<-read.csv("F:/loan_approval_dataset.csv",header = TRUE,sep=",")

data

cat("Missing values in columns:\n")

print(which(is.na(data$no_of_dependents)))

print(which(is.na(data$education)))

print(which(is.na(data$self_employed)))

print(which(is.na(data$income_annum)))

print(which(is.na(data$loan_amount)))

print(which(is.na(data$loan_term)))

print(which(is.na(data$cibil_score)))

print(which(is.na(data$residential_assets_value)))

print(which(is.na(data$commercial_assets_value)))

print(which(is.na(data$luxury_assets_value)))

print(which(is.na(data$bank_asset_value)))

print(which(is.na(data$loan_status)))

cat("Any missing values in the dataset?\n")

print(anyNA(data))

cat("Matrix of missing values:\n")

print(is.na(data))

missing_values_indices <- lapply(data, function(x) {
```

```
 if (is.integer(x) | is.character(x)) {
return(which(is.na(x) | x == ""))
 } else {
return(NULL) }})
cat("Missing values indices:\n")
print(missing_values_indices)

for (nm in names(data)) {
  if (is.character(data[[nm]]) || is.factor(data[[nm]])) {
    x <- as.character(data[[nm]])
    x[trimws(x) == ""] <- NA
    data[[nm]] <- x
  }
}
cat("Missing values per column:\n")
print(colSums(is.na(data)))
cat ("Total missing values in the dataset:\n")
print(sum(is.na(data)))

missing_counts <- sapply(missing_values_indices, length)
 barplot(
  missing_counts,
  main = "Missing Values per Column",
```

```
  xlab = "Columns",
  ylab = "Count of Missing Values",
  col = "lightblue",
  border = "darkblue",
  las = 2,
  cex.names = 0.4,
  cex.axis = 0.4
)

df <- read.csv("F:/loan_approval_dataset.csv", stringsAsFactors = FALSE)
get_mode <- function(v) {
  uniqv <- na.omit(unique(v))
  uniqv[which.max(tabulate(match(v, uniqv)))]
}

numeric_cols <- c("no_of_dependents", "income_annum", "loan_amount", "loan_term",
                  "cibil_score", "residential_assets_value", "commercial_assets_value",
                  "luxury_assets_value", "bank_asset_value")
for (col_name in numeric_cols) {
  df[[col_name]] <- as.numeric(df[[col_name]])
}
for (col_name in numeric_cols) {
  column_mode <- get_mode(df[[col_name]][!is.na(df[[col_name]])])
```

```
  df[[col_name]][is.na(df[[col_name]])] <- column_mode
  df[[col_name]] <- round(df[[col_name]], digits = 0)
}
numOfMissValue <- colSums(is.na(df[, numeric_cols]))
numOfMissValue
head(df[, numeric_cols])

numeric_cols <- c("no_of_dependents", "income_annum", "loan_amount", "loan_term",
                  "cibil_score", "residential_assets_value", "commercial_assets_value",
                  "luxury_assets_value", "bank_asset_value")

for (col_name in numeric_cols) {
  df[[col_name]] <- as.numeric(df[[col_name]])
}

for (col_name in numeric_cols) {
  mean_value <- mean(df[[col_name]], na.rm = TRUE)
  df[[col_name]][is.na(df[[col_name]])] <- mean_value
}

colSums(is.na(df[, numeric_cols]))
head(df[, numeric_cols])
```

```
numeric_cols <- c("no_of_dependents", "income_annum", "loan_amount", "loan_term",
                  "cibil_score", "residential_assets_value", "commercial_assets_value",
                  "luxury_assets_value", "bank_asset_value")
for (col_name in numeric_cols) {
  df[[col_name]] <- as.numeric(df[[col_name]])
}
cat("Missing values before:\n")
print(colSums(is.na(df[, numeric_cols])))
for (col_name in numeric_cols) {
median_value <- median(df[[col_name]], na.rm = TRUE)
  df[[col_name]][is.na(df[[col_name]])] <- median_value
}
cat("Missing values after:\n")
print(colSums(is.na(df[, numeric_cols])))
head(df[, numeric_cols])


numeric_cols <- sapply(df, is.numeric)
for(col in names(df)[numeric_cols]) {
df[[col]][df[[col]] < 0] <- NA
}
for(col in names(df)[numeric_cols]) {
  df[[col]][is.na(df[[col]])] <- median(df[[col]], na.rm = TRUE)
}
```

```
handle_outliers <- function(x) {
  Q1 <- quantile(x, 0.25)
  Q3 <- quantile(x, 0.75)
  IQR_val <- Q3 - Q1
  lower <- Q1 - 1.5 * IQR_val
  upper <- Q3 + 1.5 * IQR_val
  outliers <- x[x < lower | x > upper]
  cat("Outliers:\n")
  print(outliers)
  x[x < lower] <- lower
  x[x > upper] <- upper
  cat("\nSummary after handling:\n")
  print(summary(x))
  cat("\n===================================\n")
  return(x)
}
for(col in names(df)[numeric_cols]) {
  cat("Column:", col, "\n")
  df[[col]] <- handle_outliers(df[[col]])
}
df$loan_amount_cat <- cut(df$loan_amount,
                          breaks = quantile(df$loan_amount, probs = c(0, 0.33, 0.66, 1), na.rm = TRUE),
                          labels = c("Low", "Medium", "High"),
```

```
                    include.lowest = TRUE)
df$income_annum_cat <- cut(df$income_annum,
                     breaks = quantile(df$income_annum, probs = c(0, 0.33, 0.66, 1), na.rm = TRUE),
                     labels = c("Low", "Medium", "High"),
                     include.lowest = TRUE)
head(df[, c("loan_amount", "loan_amount_cat", "income_annum", "income_annum_cat")])


df$education <- ifelse(trimws(df$education) == "", NA, trimws(df$education))
df$self_employed <- ifelse(trimws(df$self_employed) == "", NA, trimws(df$self_employed))
df$loan_status <- ifelse(trimws(df$loan_status) == "", NA, trimws(df$loan_status))
df$education_num <- as.integer(factor(df$education, levels = c("Not Graduate", "Graduate")))
df$self_employed_num <- as.integer(factor(df$self_employed, levels = c("No", "Yes")))
df$loan_status_num <- as.integer(factor(df$loan_status, levels = c("Rejected", "Approved")))
head(df[, c("education", "education_num", "self_employed", "self_employed_num",
"loan_status", "loan_status_num")])


numeric_cols <- c("no_of_dependents", "income_annum", "loan_amount",
            "loan_term", "cibil_score", "residential_assets_value",
            "commercial_assets_value", "luxury_assets_value",
            "bank_asset_value")


normalize <- function(x) {
  return((x - min(x)) / (max(x) - min(x)))
}
```

```
normalized_data <- data
normalized_data[numeric_cols] <- lapply(data[numeric_cols], normalize)
head(normalized_data[numeric_cols])

cat("Number of duplicate rows:\n")
print(sum(duplicated(data)))
duplicates <- data[duplicated(data), ]
duplicates
data_no_duplicates <- data[!duplicated(data), ]
cat("Rows before removing duplicates:\n")
print(nrow(data))
cat("Rows after removing duplicates:\n")
print(nrow(data_no_duplicates))

high_income <- subset(data, income_annum > 5000000)
head(high_income)
long_term <- subset(data, loan_term > 10)
head(long_term)
good_cibil <- subset(data, cibil_score > 700)
head(good_cibil)
filtered_records <- subset(data, income_annum > 5000000 & cibil_score > 700)
head(filtered_records)
```

```
library(caret)

set.seed(42)

print(summary(data))

data[data == ""] <- NA

numeric_cols <- c("no_of_dependents","income_annum","loan_amount","loan_term",

                  "cibil_score","residential_assets_value","commercial_assets_value",

                  "luxury_assets_value","bank_asset_value")

for (c in numeric_cols) {

  data[[c]] <- as.numeric(data[[c]])

}

data[which(data$income_annum < 0), "income_annum"] <- NA

data[which(data$loan_amount < 0), "loan_amount"] <- NA

data[which(data$loan_term < 0), "loan_term"] <- NA

data[which(data$residential_assets_value < 0), "residential_assets_value"] <- NA

data[which(data$cibil_score < 0), "cibil_score"] <- NA


library(dplyr)

for (c in numeric_cols) {

  data[[c]][is.na(data[[c]])] <- median(data[[c]], na.rm = TRUE)

}

print(table(trimws(data$education)))

data$education <- trimws(data$education)

data$education[data$education == "Grdauate"] <- "Graduate"
```

```
data$education[data$education == "Graduate "] <- "Graduate"
print(table(data$education))
print(summary(data))

library(caret)
data$loan_status <- as.factor(data$loan_status)
set.seed(123)
balanced_over <- upSample(x = data[, -which(names(data) == "loan_status")],
                          y = data$loan_status)
balanced_over <- cbind(balanced_over, loan_status = balanced_over$Class)
balanced_over$Class <- NULL
table(balanced_over$loan_status)

library(caret)
set.seed(42)
index <- createDataPartition(data$loan_status, p = 0.8, list = FALSE)
train <- data[index, ]
test  <- data[-index, ]
print(nrow(train))
print(nrow(test))
print(table(train$loan_status))
print(table(test$loan_status))
library(dplyr)
```

```
numeric_cols <- c("income_annum","loan_amount","loan_term","cibil_score",
                  "residential_assets_value","commercial_assets_value",
                  "luxury_assets_value","bank_asset_value")
for (col in numeric_cols) {
 data[[col]] <- as.numeric(as.character(data[[col]]))
}
data$loan_status_num <- ifelse(tolower(data$loan_status) == "approved", 1, 0)
desc_stats <- data %>%
 group_by(loan_status_num) %>%
 summarise(
 Mean_Income = mean(income_annum, na.rm = TRUE),
Median_Income = median(income_annum, na.rm = TRUE),
SD_Income = sd(income_annum, na.rm = TRUE),
Min_Income = min(income_annum, na.rm = TRUE),
Max_Income = max(income_annum, na.rm = TRUE),
Mean_LoanAmount = mean(loan_amount, na.rm = TRUE),
Median_LoanAmount = median(loan_amount, na.rm = TRUE),
SD_LoanAmount = sd(loan_amount, na.rm = TRUE),
Min_LoanAmount = min(loan_amount, na.rm = TRUE),
Max_LoanAmount = max(loan_amount, na.rm = TRUE)
  )
cat("Descriptive statistics of income_annum and loan_amount by loan_status (0 = Rejected, 1 = Approved):\n")
print(desc_stats)
```

```
df$loan_status <- trimws(df$loan_status)

df$loan_status <- factor(df$loan_status, levels = c("Rejected", "Approved"))

mean_comparison <- aggregate(income_annum ~ loan_status, data = df, FUN = mean, na.rm =
TRUE)

cat("Mean income_annum by loan_status:\n")

print(mean_comparison)

library(dplyr)

df$loan_status <- trimws(df$loan_status)

df$loan_status <- factor(df$loan_status, levels = c("Rejected", "Approved"))

spread_loan <- df %>%

group_by(loan_status) %>%

summarise(

Range = max(loan_amount, na.rm = TRUE) - min(loan_amount, na.rm = TRUE),

IQR = IQR(loan_amount, na.rm = TRUE),

Variance = var(loan_amount, na.rm = TRUE),

SD = sd(loan_amount, na.rm = TRUE)

)

cat("Spread of loan_amount across loan_status:\n")

print(spread_loan)
```